DER CODE · VERSTÄNDLICH ERKLÄRT · KOLLOQUIUM

Die Stellen, auf die es ankommt

Zehn Schlüsselstellen des Codes und die 19 Codeausschnitte der Arbeit, jede mit den Zeilennummern aus dem Abgabeordner und wenigen Sätzen zum Vortragen. Die drei Code-Hefte nennen jede Funktion mit ihrem Zweck. Dieses Heft zeigt die Zeilen, auf die man im Kolloquium zeigen kann.

Ordner: prep/, tests/, vorpruefung/, modelle/
Umfang: 10 Schlüsselstellen, 19 Codeausschnitte der Arbeit
Anschluss an: Heft 1 bis 3 der Codefassung für Laien
Grundlage: der Quelltext im Abgabeordner und die Abgabe-PDF, Stand 11.10.2026

Heft 1
prep/ und tests/

Heft 2
vorpruefung/

Heft 3
modelle/

Kolloquium
Stellen zum Zeigen

TEIL 1

Die Stellen auf einen Blick

> Im Kolloquium kann jede Zeile des Abgabeordners zur Frage werden. Dieses Heft sammelt die Stellen, die man zeigen können sollte, und sagt zu jeder in wenigen Sätzen, was dort passiert.

Das Heft hat zwei Teile. Teil 2 zeigt zehn Stellen, an denen der Code die wichtigsten Entscheidungen der Arbeit trifft und die in der Arbeit nicht als Ausschnitt abgedruckt sind. Teil 3 zeigt die 19 Codeausschnitte der Arbeit so, wie sie gedruckt sind, mit denselben Zeilennummern und der Seite in der Arbeit. Zusammen reichen sie von den Rohdaten bis zur Erklärung der Ergebnisse.

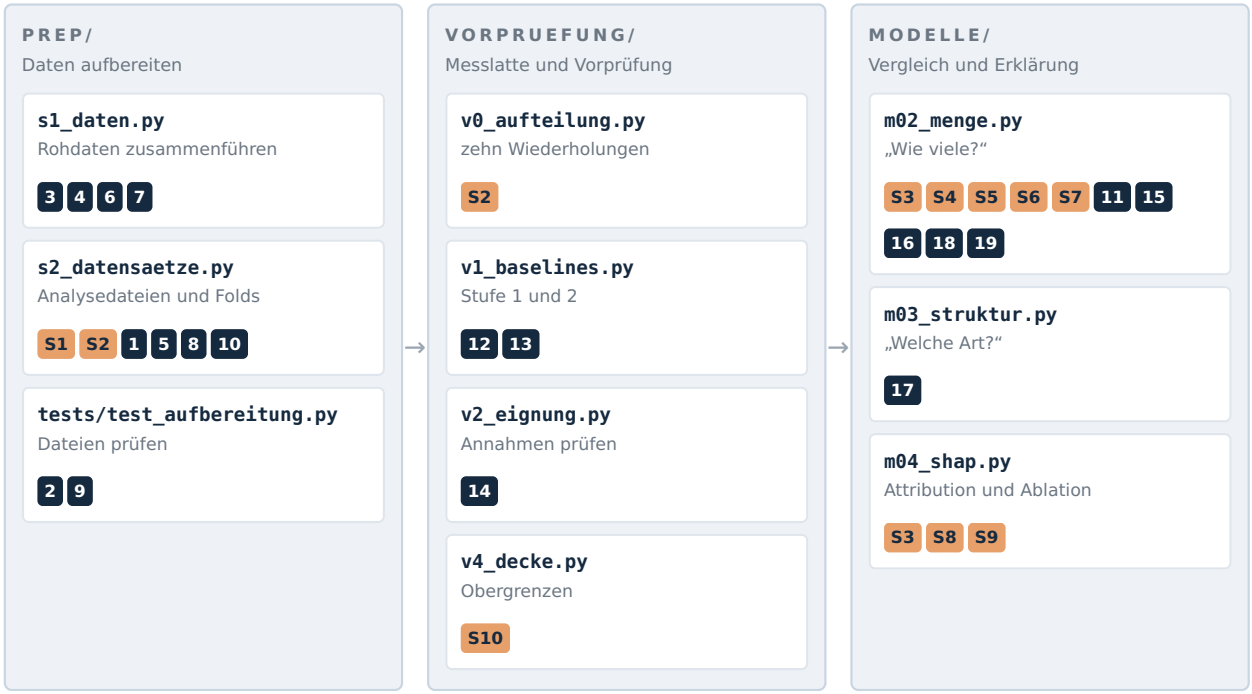


Abbildung 1: Wo die Stellen liegen. Orange sind die zehn Stellen aus Teil 2, dunkelblau die Codeausschnitte der Arbeit aus Teil 3. Jede Marke führt im PDF zur Stelle.

SO STELLT MAN EINE STELLE VOR

Erstens die Frage, die die Stelle beantwortet. Zweitens die Zeile, an der man es sieht. Drittens die Folge für das Ergebnis. Der fett gesetzte Satz unter jedem Ausschnitt ist die Antwort in einem Satz, der Text darunter führt durch die Zeilen.

Alle Zeilennummern sind die des Quelltexts im Abgabeordner. Eine Datei lässt sich also direkt an derselben Stelle öffnen. Wo Zeilen ausgelassen sind, steht ⋮ in der Nummernspalte.

Die zehn Stellen aus Teil 2

NR.	STELLE	DATEI	SEITE
S1	Die Einsatzart als Zielgröße	prep/s2_datensaetze.py	4
S2	Zehn Wiederholungen derselben Aufteilung	vorpruefung/v0_aufteilung.py prep/s2_datensaetze.py	5
S3	Was das Training nicht gesehen hat	modelle/m02_menge.py modelle/m04_shap.py	6
S4	Der gepaarte Vergleich	modelle/m02_menge.py	7
S5	Zwei Teststufen und die Korrektur nach Holm	modelle/m02_menge.py	8
S6	Leckt die Suche aus Wiederholung 0?	modelle/m02_menge.py	9
S7	Das Hold-out bleibt bis zum Schluss gesperrt	modelle/m02_menge.py	10
S8	Attribution mit SHAP	modelle/m04_shap.py	11
S9	Ablation der Faktorgruppen	modelle/m04_shap.py	12
S10	Wie viel Macro-F1 überhaupt erreichbar ist	vorpruefung/v4_decke.py	13

Die 19 Codeausschnitte der Arbeit aus Teil 3

CODE	TITEL IN DER ARBEIT	DATEI	ARBEIT	HEFT
1	Prüfung von Zuschnitt und Wohnbevölkerung	prep/s2_datensaetze.py	29	15
2	Prüfung auf Ergebnisvariablen in der erzeugten Datei	tests/test_aufbereitung.py	29	15
3	Zuordnung über Zensusgrenzen hinweg	prep/s1_daten.py	31	16
4	Zeitbewusste Zuordnung des ACS-Jahrgangs	prep/s1_daten.py	32	16
5	Vollständiges Raster und Vorwärtsfüllen	prep/s2_datensaetze.py	33	17
6	Anteilswerte aus Zählvariablen	prep/s1_daten.py	34	17
7	Relativer Kriminalitätsindex je Stadtteil und Monat	prep/s1_daten.py	35	18
8	Der Kalendermonat als Sinus und Kosinus	prep/s2_datensaetze.py	36	18
9	Prüfung der Stadtteiltrennung an der erzeugten Datei	tests/test_aufbereitung.py	39	19
10	Fold-Zuteilung mit doppelter Stratifizierung	prep/s2_datensaetze.py	40	19
11	Mittelwert und Streuung der Gütemaße über Folds und Wiederholungen	modelle/m02_menge.py	42	20
12	Der Offset im Poisson-Modell der Stufe 2	vorpruefung/v1_baselines.py	43	20
13	Beide Baseline-Stufen des Mengenstrangs	vorpruefung/v1_baselines.py	43	21
14	Überdispersionstest als Hilfsregression	vorpruefung/v2_eignung.py	48	21
15	Ein Lauf: Anpassung auf der Rate, Rückrechnung und Zeitmessung	modelle/m02_menge.py	51	22
16	Verlustfunktion und Zieltransformation je Verfahren	modelle/m02_menge.py	53	22
17	Globale Klassenkodierung und ausgeglichene Gewichte	modelle/m03_struktur.py	54	23
18	Innere Suche, nach Stadtteilen gruppiert	modelle/m02_menge.py	54	23
19	Der Bewertungslauf über alle Wiederholungen und Folds	modelle/m02_menge.py	55	24

TEIL 2

Zehn Stellen, auf die es ankommt

> Diese Stellen stehen nicht als Ausschnitt in der Arbeit, tragen aber ihre wichtigsten Entscheidungen. Sie zeigen, wie die Zielgröße entsteht, wie verglichen und geprüft wird, wie die Ergebnisse erklärt werden und wo ihre Grenzen liegen.

S1 prep/s2_datensaetze.py · Z. 334–386, gekürzt

baue_klassifikation()

Die Einsatzart als Zielgröße

Woraus entsteht die Klasse, die der Strukturstrang vorhersagt?

```
334 def baue_klassifikation(regression: pd.DataFrame,
335                         verbose: bool = False) -> pd.DataFrame:
359     serie = df["einsatzart"].astype(str).str.extract(r"^(\d)")[0]
360     df["gruppe"] = serie.map(NFIRS_GRUPPEN).fillna(RESTKLASSE)
362     zaehlung = (df.groupby(["stadtteil", "jahr_monat", "gruppe"]).size()
363                   .unstack(fill_value=0).reindex(columns=KLASSEN, fill_value=0)
364                   .rename(columns={k: f"anzahl_{SUFFIX[k]}" for k in KLASSEN})
365                   .reset_index())
374     ohne = d[ZIELGROESSE] == 0
375     if ohne.any():
376         d = d[~ohne].reset_index(drop=True)
378     for k in KLASSEN:
379         d[f"anteil_{SUFFIX[k]}"] = d[f"anzahl_{SUFFIX[k]}"] / d[ZIELGROESSE]
385     d[ZIELKLASSE] = d[ANTEILE].idxmax(axis=1).str.replace("anteil_", "",
386                                                           regex=False)
```

KURZ GESAGT

Jeder Einsatz erhält über die erste Ziffer seines NFIRS-Codes eine von vier Gruppen. Je Stadtteil und Monat gewinnt die Gruppe mit dem größten Anteil.

Zeile 359 schneidet die führende Ziffer ab und Zeile 360 übersetzt sie über `NFIRS_GRUPPEN` in eine Gruppe. Codes ohne Zuordnung landen in der Restklasse technische Hilfe. Die Zählung in den Zeilen 362 bis 365 legt mit `reindex` die Spalten fest, sodass auch eine Gruppe ohne Einsatz als Null erscheint.

Monate ohne jeden Einsatz fallen in Zeile 376 heraus, weil ihr Anteil 0/0 wäre. Deshalb hat der Strukturdatensatz 4.751 statt 4.752 Zeilen. Zeile 385 nimmt mit `idxmax` den größten Anteil. Bei Gleichstand gewinnt die erste Spalte, und die Spalten laufen von Brand bis Fehlalarm, also von der seltensten zur häufigsten Gruppe.

S2 vorpruefung/v0_aufteilung.py · Z. 62–108, gekürzt

wiederholte_aufteilung() · fold_masken()

Zehn Wiederholungen derselben Aufteilung

Wie entstehen zehn verschiedene Fold-Zuteilungen, ohne dass das Hold-out wandert?

```python
def wiederholte_aufteilung(daten: pd.DataFrame, wiederholung: int = 0,
                           selten: pd.Series | None = None) -> pd.DataFrame:

    entwicklung = daten[daten["ist_holdout"] == 0]
    bev = entwicklung.groupby("stadtteil")[EXPOSURE_ROH].mean()

    ordnung = list(pd.DataFrame({"selten": s, "bev": bev})
                   .sort_values(["selten", "bev"], ascending=False).index)

    if wiederholung:
        rng = np.random.default_rng(RANDOM_STATE + wiederholung)
        for anfang in range(0, len(ordnung), N_FOLDS):
            block = ordnung[anfang:anfang + N_FOLDS]
            ordnung[anfang:anfang + N_FOLDS] = [block[i] for i in
                                                rng.permutation(len(block))]

    gruppe = {st: i % N_FOLDS + 1 for i, st in enumerate(ordnung)}

    d["fold"] = d["stadtteil"].map(gruppe).fillna(0).astype("int64")
    d["ist_holdout"] = daten["ist_holdout"].astype("int64")

    if wiederholung == 0:
        abweichend = int((d["fold"] != daten["fold"]).sum())
        assert not abweichend, (
            f"Wiederholung 0 weicht in {abweichend} Zeilen von der fold-Spalte "
            f"der Datei ab - die Stratifizierung passt nicht. Wurde `selten` "
            f"uebergeben?")
```

AUS DER SPALTE WERDEN TRAININGS- UND TESTMASKE prep/s2_datensaetze.py · Z. 97–111, gekürzt

```python
def fold_masken(daten: pd.DataFrame, k: int) -> tuple[pd.Series, pd.Series]:

    test = daten["fold"] == k
    assert test.any(), (f"Kein Fold {k} im Datensatz "
                        f"(vorhanden: {sorted(daten['fold'].unique())}).")
    train = (daten["fold"] != k) & (daten["ist_holdout"] == 0)
    return train, test
```

KURZ GESAGT

Gemischt wird nur innerhalb von Fünferblöcken der sortierten Stadtteile. So bleibt jeder Fold geschichtet, und das Hold-out bleibt fest.

Zeile 80 nimmt nur die 30 Entwicklungsstadtteile. Die Zeilen 86 und 87 sortieren sie wie in der Datei, erst nach brand-dominierten Monaten, dann nach Wohnbevölkerung. Ab Wiederholung 1 vertauschen die Zeilen 94 und 95 die Stadtteile innerhalb jedes Blocks aus fünf Nachbarn der Sortierung, mit dem Startwert 42 plus Wiederholungsnummer (Zeile 91). Zeile 97 verteilt danach reihum auf die Folds 1 bis 5. Jeder Fold erhält damit aus jedem Block genau einen Stadtteil.

Die Hold-out-Stadtteile kommen in dieser Zuordnung nicht vor und erhalten in Zeile 100 die Null. Wiederholung 0 bleibt ungemischt und muss die Spalte der Datei Zeile für Zeile treffen, sonst bricht Zeile 105 ab. `fold_masken()` macht aus der Spalte die beiden Masken: Test ist der eine Fold, Training sind alle übrigen ohne Hold-out (Zeile 110).

S3 modelle/m02_menge.py · Z. 292–304, gekürzt

extrapolationsanteil() · ruhigster_fold()

Was das Training nicht gesehen hat

Wie viele Testzeilen liegen außerhalb dessen, was ein Modell gelernt hat?

```
292 def extrapolationsanteil(train: pd.DataFrame, test: pd.DataFrame) -> float:
    ⋮
302     lo, hi = train[MERKMALE].min(), train[MERKMALE].max()
303     aussen = ((test[MERKMALE] < lo) | (test[MERKMALE] > hi)).any(axis=1)
304     return float(aussen.mean())
```

DIESELBE ZAHL WÄHLT DEN FOLD FÜR DIE ATTRIBUTION modelle/m04_shap.py · Z. 91–99, gekürzt

```
91 def ruhigster_fold(folds: pd.DataFrame) -> int:
   ⋮
97     w0 = folds[folds["wiederholung"] == 0]
98     je_fold = w0.groupby("fold")["extrapolationsanteil"].first()
99     return int(je_fold.idxmin())
```

KURZ GESAGT

Gezählt wird, wie viele Testzeilen in mindestens einem Merkmal unter dem Minimum oder über dem Maximum der Trainingszeilen liegen.

Zeile 302 nimmt je Merkmal Minimum und Maximum der Trainingsstadtteile. Zeile 303 markiert eine Testzeile, sobald eines ihrer zwölf Merkmale darunter oder darüber liegt, und Zeile 304 gibt den Anteil zurück. In der ersten Wiederholung liegt er je Fold zwischen 0,0 und 66,7 %. Ein Baum gibt für solche Zeilen den Wert seiner Randregion aus.

`ruhigster_fold()` in m04 wählt mit derselben Zahl den Fold für die Attribution. Das ist Fold 5 der ersten Wiederholung, der einzige ohne Testwert außerhalb des gelernten Bereichs. Gemessen wird nur die Spanne je Merkmal. Neue Kombinationen bekannter Werte zählen nicht mit, das echte Problem ist also eher größer.

S4 modelle/m02_menge.py · Z. 504–581, gekürzt

_gepaart() · paar()

Der gepaarte Vergleich

Wie wird geprüft, ob ein Verfahren besser ist als ein anderes?

```
504 def _gepaart(a: np.ndarray, b: np.ndarray) -> dict:
514     diff = np.asarray(b, float) - np.asarray(a, float)
515     n = len(diff)
516     mittel = float(diff.mean())
517     if n > 1 and diff.std(ddof=1) > 0:
518         halb = float(t.ppf(1 - ALPHA / 2, n - 1) * diff.std(ddof=1) / np.sqrt(n))
519     else:
520         halb = 0.0
521     try:
522         p = float(wilcoxon(diff, zero_method="wilcox").pvalue)
523     except ValueError:            # alle Differenzen null
524         p = 1.0
525     return {"n_paare": n, "differenz_mittel": mittel,
526             "ci_unten": mittel - halb, "ci_oben": mittel + halb,
527             "gewonnene": int((diff > 0).sum()), "wilcoxon_p": p}
568             def paar(links: pd.Series, rechts: pd.Series) -> dict:
574                 zusammen = pd.concat([links.rename("a"), rechts.rename("b")],
575                                      axis=1, join="inner")
576                 fehlend = max(len(links), len(rechts)) - len(zusammen)
577                 assert not fehlend, (
578                     f"{fehlend} Laeufe ohne Gegenstueck bei {ziel}, Stufe "
579                     f"{stufe} - Verfahren und Baseline liefen auf "
580                     f"unterschiedlichen Aufteilungen.")
581                 return _gepaart(zusammen["a"].to_numpy(), zusammen["b"].to_numpy())
```

KURZ GESAGT

Verglichen wird nicht Mittelwert gegen Mittelwert, sondern Lauf gegen Lauf auf denselben Teststadtteilen.

Zeile 514 bildet je Paar die Differenz b minus a. Beim RMSE ist kleiner besser, eine positive Differenz ist also ein Vorteil für a. Die Zeilen 517 und 518 legen um die mittlere Differenz ein 95-%-Intervall nach der t-Verteilung. Zeile 522 rechnet den Vorzeichen-Rang-Test nach Wilcoxon, Zeile 527 zählt, wie oft a vorn liegt. Diese Zahl braucht keine Annahme über die Verteilung.

Wer mit wem gepaart wird, entscheidet `paar()` in den Zeilen 574 und 575 über die Schlüssel Wiederholung und Fold und nicht über die Reihenfolge der Zeilen. Fehlt ein Gegenstück, bricht Zeile 577 ab, denn dann liefen Verfahren und Baseline auf verschiedenen Aufteilungen. In m03 ist beim Macro-F1 größer besser, dort dreht die Aufrufstelle die beiden Argumente.

S5 modelle/m02_menge.py · Z. 484–607, gekürzt

vergleiche() · _holm()

Zwei Teststufen und die Korrektur nach Holm

Welche Tests werden korrigiert und auf welchen Zahlen laufen sie?

```
484 def _holm(p: np.ndarray) -> np.ndarray:
494     m = len(p)
495     ordnung = np.argsort(p)
496     angepasst = np.empty(m, float)
497     laufend = 0.0
498     for rang, i in enumerate(ordnung):
499         laufend = max(laufend, (m - rang) * p[i])
500         angepasst[i] = min(laufend, 1.0)
501     return angepasst
552     for stufe, schluessel in (("wiederholung", ["wiederholung"]),
553                               ("lauf", ["wiederholung", "fold"])):
556         v = (folds.groupby(["zielgroesse", "verfahren"] + schluessel,
557                            sort=False)[TESTMASS].mean().rename("wert").reset_index())
602     df["p_holm"] = np.nan
603     for stufe in df["teststufe"].unique():
604         maske = (df["rolle"] == "sekundaer") & (df["teststufe"] == stufe)
605         df.loc[maske, "p_holm"] = _holm(df.loc[maske, "wilcoxon_p"].to_numpy())
606     df["signifikant"] = np.where(
607         df["rolle"] == "primaer", df["wilcoxon_p"] < ALPHA, df["p_holm"] < ALPHA)
```

KURZ GESAGT

Die Vergleiche mit der Referenz laufen einzeln. Nur die Vergleiche der Verfahren untereinander bilden eine Familie und werden nach Holm korrigiert.

Die Schleife in Zeile 552 rechnet jeden Test zweimal: auf den zehn Wiederholungsmitteln als Haupttest und auf den 50 einzelnen Läufen als gekennzeichnete Gegenprobe. Für die erste Stufe mitteln die Zeilen 556 und 557 zuerst die fünf Folds jeder Wiederholung. Die 50 Läufe sind dieselben 30 Stadtteile in zehn Gruppierungen und lieferten zu kleine p-Werte.

`_holm()` sortiert die p-Werte aufsteigend und multipliziert den kleinsten mit m, den nächsten mit m minus eins (Zeile 499). Das laufende Maximum hält die Reihenfolge, Zeile 500 kappt bei eins. Zeile 605 wendet die Korrektur nur auf die Rolle `sekundaer` an, im Mengenstrang sechs Tests aus drei Paarungen und zwei Zielgrößen. Zeile 606 entscheidet danach für die Vergleiche mit der Referenz am rohen und für die übrigen am korrigierten p-Wert.

S6 modelle/m02_menge.py · Z. 613–644, gekürzt

leakage_diagnose()

Leckt die Suche aus Wiederholung 0?

Profitieren die Wiederholungen 1 bis 9 davon, dass nur in Wiederholung 0 gesucht wurde?

```
613 def leakage_diagnose(folds: pd.DataFrame, baselines: pd.DataFrame) -> pd.DataFrame:
    ⋮
629     basis = (baselines[baselines["modell"] == BASELINE_STUFE2]
630              .set_index(["zielgroesse", "wiederholung", "fold"])[TESTMASS])
    ⋮
632     for (ziel, name), g in folds.groupby(["zielgroesse", "verfahren"], sort=False):
633         g = g.set_index(["zielgroesse", "wiederholung", "fold"])
    ⋮
635         vorsprung = basis.reindex(g.index) - g[TESTMASS]
636         w0 = vorsprung.xs(0, level="wiederholung")
637         rest = vorsprung[vorsprung.index.get_level_values("wiederholung") > 0]
638         zeilen.append({
639             "zielgroesse": ziel, "verfahren": name, "mass": TESTMASS,
640             "vorsprung_w0": float(w0.mean()), "n_w0": int(len(w0)),
641             "vorsprung_w1_9": float(rest.mean()), "n_w1_9": int(len(rest)),
642             "differenz": float(rest.mean() - w0.mean()),
643             "differenz_in_std_folds": float((rest.mean() - w0.mean())
644                                             / g[TESTMASS].std())})
```

KURZ GESAGT

Gesucht wurde nur in Wiederholung 0. Die Diagnose vergleicht den Vorsprung auf die Baseline dort mit dem in den neun übrigen Wiederholungen.

Zeile 635 bildet je Lauf den Vorsprung als RMSE der Baseline minus RMSE des Verfahrens. Zeile 636 nimmt die fünf Läufe der Wiederholung 0, in der jeder Fold die Parameter aus seiner eigenen Trainingsmenge hat. Zeile 637 nimmt die 45 Läufe der Wiederholungen 1 bis 9. Dort lagen im Mittel 80 % der Teststadtteile in der Menge, auf der gesucht wurde. Wäre das ein Leck, müsste der Vorsprung dort größer sein.

Zeile 643 setzt den Unterschied ins Verhältnis zur Streuung über die Folds. In allen acht Vergleichen beider Stränge fällt der Vorsprung kleiner aus, um das 0,098- bis 0,317-Fache dieser Streuung. Die Diagnose ist bewusst schwach, denn auch Wiederholung 0 ist eine eigene Aufteilung. Ein großer Effekt wäre sichtbar, ein kleiner nicht von der Schwankung zu trennen.

S7 modelle/m02_menge.py · Z. 794–796

main() · hold_out()

Das Hold-out bleibt bis zum Schluss gesperrt

Wo wird verhindert, dass die sechs zurückgehaltenen Stadtteile zu früh ins Spiel kommen?

DIE SPERRE STEHT VOR ALLEM ANDEREN modelle/m02_menge.py · Z. 794–796

```
794 # Fallstrick 3: ohne "holdout" hier auf die Entwicklungsstadtteile
795 # einschraenken; danach sieht nichts mehr Hold-out-Zeilen.
796 panel = voll[voll["ist_holdout"] == 0].reset_index(drop=True)
```

DIE EINMALIGE SCHLUSSBEWERTUNG modelle/m02_menge.py · Z. 653–706, gekürzt

```
653 def hold_out(panel: pd.DataFrame, parameter: pd.DataFrame,
654                folds: pd.DataFrame, selten: pd.Series) -> pd.DataFrame:
668     param = _parameter_je_fold(parameter)
669     dev, ho = entwicklung_und_holdout(panel)
670     train, test = panel[dev], panel[ho]
681     anzahl = poisson_glm(train, test)
699     w0 = folds[folds["wiederholung"] == 0]
700     for ziel in ZIELE:
701         for name in VERFAHREN:
702             g = w0[(w0["zielgroesse"] == ziel) & (w0["verfahren"] == name)]
703             bester = int(g.loc[g["RMSE"].idxmin(), "fold"])
704             z = ein_lauf(name, param[(ziel, name, bester)], train, test, ziel)
705             zeilen.append({**z, "stufe": 3, "fold_der_parameter": bester,
706                            "n_stadtteile_test": int(test["stadtteil"].nunique())})
```

KURZ GESAGT

Die sechs Hold-out-Stadtteile fallen in Zeile 796 aus dem Panel, bevor irgendetwas rechnet. Bewertet werden sie genau einmal am Ende.

Tuning, Bewertung und Vergleich sehen nur das zugeschnittene Panel. Erst mit dem Argument `holdout` ruft `main()` die Funktion `hold_out()` mit dem vollen Datensatz auf. Zeile 669 trennt dort über `entwicklung_und_holdout()` aus v0: Trainiert wird auf allen 30 Entwicklungsstadtteilen, bewertet auf den sechs zurückgehaltenen. Die Poisson-Referenz läuft in Zeile 681 mit.

Welchen der fünf Parametersätze ein Verfahren bekommt, entscheidet Zeile 703: den des Folds mit dem kleinsten RMSE in Wiederholung 0. Die Wahl stützt sich damit nur auf Entwicklungsdaten. m03 macht dasselbe mit dem höchsten Macro-F1. Das Ergebnis ist eine Messung an sechs Einheiten, ohne Streuung und ohne Test.

S8 modelle/m04_shap.py · Z. 65–81, gekürzt

schlagen_die_latte() · _beitraege()

Attribution mit SHAP

Welches Modell wird erklärt und wie entstehen die Beiträge je Merkmal?

ERKLÄRT WIRD NUR, WAS DIE LATTE SCHLÄGT modelle/m04_shap.py · Z. 65–81, gekürzt

```
65 def schlagen_die_latte(vergleich: pd.DataFrame) -> tuple[list, pd.DataFrame]:
   ⋮
76     p = vergleich[(vergleich["rolle"] == "primaer")
77                   & (vergleich["teststufe"] == "wiederholung")].copy()
78     p["verfahren"] = p["paarung"].str.split(" vs ").str[0]
79     p["schlaegt"] = (p["differenz_mittel"] > 0) & p["signifikant"]
80     genommen = [(z["zielgroesse"], z["verfahren"])
81                 for _, z in p[p["schlaegt"]].iterrows()]
```

BEITRAG JE MERKMAL modelle/m04_shap.py · Z. 102–136, gekürzt

```
102 def _beitraege(modell, X: pd.DataFrame, name: str) -> np.ndarray:
    ⋮
118     if name == "ridge":
119         return np.abs(modell[-1].regressor_.coef_).ravel()
120
121     if name == "xgboost":
122         import xgboost as xgb
123         roh = modell.get_booster().predict(
124             xgb.DMatrix(X, feature_names=list(X.columns)), pred_contribs=True)
125         werte = np.abs(np.asarray(roh))[..., :-1]     # letzte Spalte = Bias
126     else:
127         import shap
128         werte = shap.TreeExplainer(modell).shap_values(X)
129         if isinstance(werte, list):                   # aeltere shap-Fassungen
130             werte = np.stack(werte, axis=-1)
131         werte = np.abs(np.asarray(werte))
132
133     if werte.ndim == 3:            # (n, klassen, p) oder (n, p, klassen)
134         achse = 1 if werte.shape[1] != len(X.columns) else 2
135         werte = werte.mean(axis=achse)
136     return werte.mean(axis=0)
```

KURZ GESAGT

Erklärt wird nur ein Modell, das die Referenz signifikant schlägt. Das ist allein der Random Forest im Strukturstrang.

Zeile 79 verlangt beides: eine mittlere Differenz zugunsten des Verfahrens und einen signifikanten Primärtest. Im Mengenstrang besteht kein Modell, dort rechnet m04 die Beiträge aus den standardisierten Koeffizienten des Poisson-GLM.

`_beitraege()` liefert je Merkmal den mittleren Betrag des Beitrags. Für Ridge sind das die Koeffizienten (Zeile 119), für den Random Forest rechnet `shap.TreeExplainer` exakt (Zeile 128). XGBoost bringt TreeSHAP selbst mit und liefert die Werte über `pred_contribs`, weil der Explainer den mehrklassigen Basiswert von XGBoost 3.x nicht lesen kann. Zeile 125 lässt dabei die letzte Spalte weg, den Basiswert. Zeile 135 mittelt über die vier Klassen, Zeile 136 über die Zeilen. `main()` normiert die Werte danach auf eins und summiert sie zu den fünf Faktorgruppen.

S9 modelle/m04_shap.py · Z. 240–289, gekürzt

ablation_faktorgruppen() · _ablation_auswerten()

Ablation der Faktorgruppen

Was kostet es, wenn eine Faktorgruppe fehlt?

```
240 def ablation_faktorgruppen(reg: pd.DataFrame, kl: pd.DataFrame,
241                           selten: pd.Series, tuning_kl: pd.DataFrame | None,
242                           mit_baeumen: bool = True) -> pd.DataFrame:
    ⋮
271     varianten = ["voll"] + list(GRUPPEN)
    ⋮
275     for w in range(WIEDERHOLUNGEN):
276         d = wiederholte_aufteilung(reg, wiederholung=w, selten=selten)
277         for k in range(1, N_FOLDS + 1):
278             tr, te = fold_masken(d, k)
279             train, test = d[tr], d[te]
280             y = test[ZIELGROESSE].astype(float).to_numpy()
281             for variante in varianten:
282                 weg = GRUPPEN.get(variante, [])
283                 merkmale = [m for m in MERKMALE if m not in weg]
284                 y_hat = poisson_glm(train, test, merkmale)
285                 zeilen.append({"strang": "menge", "verfahren": "Poisson-GLM",
286                                "weggelassen": variante, "n_merkmale": len(merkmale),
287                                "wiederholung": w, "fold": k,
288                                "mass": "RMSE",
289                                "wert": bewerte_regression(y, y_hat)["RMSE"]})
```

VORZEICHEN: POSITIV HEISST SCHLECHTER OHNE DIE GRUPPE modelle/m04_shap.py · Z. 336–357, gekürzt

```
336 def _ablation_auswerten(roh: pd.DataFrame) -> pd.DataFrame:
    ⋮
355     # RMSE: Verschlechterung = variante - voll. Macro-F1: umgekehrt.
356     schlechter = np.where(d["mass"] == "RMSE", 1.0, -1.0)
357     d["verschlechterung"] = (d["wert"] - d["voll"]) * schlechter
```

KURZ GESAGT

Jede Faktorgruppe wird einmal weggelassen und das Modell ohne sie neu angepasst, auf denselben 50 Läufen wie im Vergleich.

Zeile 271 legt sechs Varianten an, den vollen Satz und je eine ohne eine der fünf Gruppen. Die Zeilen 282 und 283 streichen die Merkmale der Gruppe, Zeile 284 passt das Poisson-GLM ohne sie neu an. Der Offset bleibt in jeder Variante bestehen, weggelassen wird nur der Prädiktor `log_bevoelkerung`. Weil das GLM keinen Hyperparameter hat, ändert sich allein die Merkmalsmenge.

`_ablation_auswerten()` paart jede Variante mit dem vollen Satz desselben Laufs und dreht in Zeile 356 das Vorzeichen, sodass ein positiver Wert immer eine Verschlechterung bedeutet. Ohne den Kriminalitätsindex steigt der RMSE um 6,10, in zehn von zehn Wiederholungen. Ohne die baulichen Merkmale sinkt er um 8,04. Im Strukturstrang laufen Logit, Random Forest und XGBoost nach demselben Muster.

S10 vorpruefung/v4_decke.py · Z. 67–95, gekürzt

decke_a() · decke_b()

Wie viel Macro-F1 überhaupt erreichbar ist

Wo liegt die Obergrenze für die Einsatzart, bevor ein Modell rechnet?

DECKE A: UNSCHÄRFE DES LABELS vorpruefung/v4_decke.py · Z. 67–95, gekürzt

```
67  def decke_a(panel: pd.DataFrame) -> tuple[float, float, float]:
    ⋮
82      zaehler = panel[ZAEHLER].to_numpy(dtype=float)
83      n = zaehler.sum(axis=1).astype(int)
84      if (n == 0).any():
85          zaehler, n = zaehler[n > 0], n[n > 0]
86      p = zaehler / n[:, None]
87      beobachtet = zaehler.argmax(axis=1)
88
89      rng = np.random.default_rng(RANDOM_STATE)
90      werte, kipp = [], np.zeros(len(n))
91      for _ in range(ZIEHUNGEN):
92          gezogen = np.array([rng.multinomial(k, q) for k, q in zip(n, p)]).argmax(axis=1)
93          werte.append(_macro_f1(beobachtet, gezogen))
94          kipp += gezogen != beobachtet
95      return float(np.mean(werte)), float(np.std(werte, ddof=1)), float((kipp / ZIEHUNGEN).mean())
```

DECKE B: WISSEN ÜBER DEN STADTTEIL vorpruefung/v4_decke.py · Z. 98–112, gekürzt

```
98  def decke_b(panel: pd.DataFrame) -> tuple[float, float, pd.Series]:
    ⋮
109     modal = panel.groupby("stadtteil")[ZIEL].agg(lambda s: s.mode().iloc[0])
110     vorhersage = panel["stadtteil"].map(modal)
111     treffer = float((vorhersage == panel[ZIEL]).mean())
112     return _macro_f1(panel[ZIEL], vorhersage), treffer, modal.value_counts()
```

KURZ GESAGT

Zwei Obergrenzen stehen fest, bevor ein Modell rechnet. Sie zeigen, wie viel Macro-F1 mit dieser Zielgröße und diesen Merkmalen überhaupt drin ist.

Decke A fragt, wie scharf das Label selbst ist. Zeile 86 bildet je Stadtteilmonat die beobachteten Anteile, Zeile 92 zieht die Einsätze 200-mal neu aus einer Multinomialverteilung und nimmt jeweils den größten Anteil. Der Macro-F1 zwischen altem und neuem Label in Zeile 93 ist die Güte eines Modells, das die wahren Anteile kennt. Im Entwicklungspanel ergibt das 0,641, bei einer neuen Ziehung wechseln im Mittel 13,6 % der Monate ihre Klasse.

Decke B bindet enger. Zeile 109 bestimmt für jeden Stadtteil seine häufigste Klasse, Zeile 110 sagt sie für alle seine Monate vorher. Mehr kann ein Modell aus Merkmalen, die sich im Stadtteil kaum ändern, im Kern nicht ableiten, und 27 der 30 Entwicklungsstadtteile haben dieselbe häufigste Klasse. Das ergibt 0,415. Der Random Forest schließt 50,2 % des Abstands zwischen Mehrheitsklasse und dieser Decke.

Sechs weitere Stellen

Falls danach gefragt wird

Ohne Ausschnitt, mit Datei und Zeilen zum schnellen Aufschlagen.

DATEI · ZEILEN	FUNKTION	WAS DORT PASSIERT
`modelle/trennschaerfe.py` Z. 44–63	`effektstaerke()` `trennschaerfe()`	Rechnet aus Differenz und Intervall die Effektstärke d und simuliert mit 4.000 Ziehungen, wie oft der Wilcoxon-Test bei zehn Paaren verwirft. Bei d = 1,00 sind es 78 %.
`modelle/parametersensitivitaet.py` Z. 147–195	`kreuzprobe()`	Wertet jeden Testfold mit jedem der fünf Parametersätze aus. Der eigene Satz ist nicht systematisch besser als ein fremder.
`modelle/suchdiagnose.py` Z. 86–118, 134–146	`eine_suche()` `kurve()`	Wiederholt die Suche, schreibt alle 100 Ziehungen mit und bildet das laufende Maximum. Die zweite Hälfte verbessert den besten Wert im Mittel um höchstens 0,020.
`vorpruefung/v3_spezifikation.py` Z. 62–101, 174–198	`entwerfe()` `_selbsttest()`	Baut Quadrate und Interaktionen auf Merkmalen, die nur mit Trainingswerten standardisiert sind. Der Selbsttest verlangt, dass die lineare Variante die Stufe-2-Referenz trifft.
`modelle/m02_menge.py` Z. 236–251	`ein_lauf()`, `Parallelmessung`	Passt jedes Modell ein zweites Mal über alle Kerne an und hält die größte Abweichung fest. Bei XGBoost sind es bis zu 50,88 Einsätze.
`modelle/m03_struktur.py` Z. 327–347	`_macro_auroc()`	Gibt einen fehlenden Wert statt 0,5 zurück, wenn im Testfold eine Klasse fehlt. Ein erfundener Wert sähe aus wie eine Messung.

TEIL 3

Die 19 Codeausschnitte der Arbeit

> Die Ausschnitte stehen hier so, wie sie in der Arbeit gedruckt sind, gekürzt und mit den Zeilennummern des Quelltexts. Alle Zeilen sind gegen den Abgabeordner geprüft und stimmen mit ihm überein.

CODE 1 · SEITE 29 DER ARBEIT

Prüfung von Zuschnitt und Wohnbevölkerung

`prep/s2_datensaetze.py` · Z. 415–440, gekürzt · Quelle in der Arbeit: Claude2026w

```
415 def pruefe_zuschnitt(r: pd.DataFrame) -> None:
    ⋮
428     ist = sorted(r["stadtteil"].unique())
429     alle = sorted(pd.read_parquet(PFAD_EINSAETZE, columns=["stadtteil"])
430                   ["stadtteil"].dropna().unique())
431     assert len(ist) == N_STADTTEILE_ERWARTET, (
432         f"{len(ist)} Analyseeinheiten statt {N_STADTTEILE_ERWARTET}. "
433         f"Nicht enthalten: {sorted(set(alle) - set(ist))}")
    ⋮
435     bev = (r.groupby(["jahr", "stadtteil"])[EXPOSURE_ROH].first()
436             .groupby("jahr").sum())
437     unten, oben = BEV_PLAUSIBEL
438     assert unten <= bev.min() and bev.max() <= oben, (
    ⋮
440         f"ausserhalb {unten:,}-{oben:,} - ACS-Zuordnung pruefen "
```

KURZ GESAGT **Nach der Aufbereitung prüft die Funktion zwei Größen, die ein stiller Datenverlust zwangsläufig bewegt.**

Die erste Bedingung in Zeile 431 verlangt genau 36 Stadtteile und nennt im Fehlerfall die fehlenden (Zeile 433). Die zweite summiert je Jahr die Wohnbevölkerung und verlangt in Zeile 438 einen Wert zwischen 750.000 und 950.000. Sie fängt den Fall, in dem ein Stadtteil vorhanden ist, ihm aber einzelne Tracts fehlen.

CODE 2 · SEITE 29 DER ARBEIT

Prüfung auf Ergebnisvariablen in der erzeugten Datei

`tests/test_aufbereitung.py` · Z. 279–287, gekürzt · Quelle in der Arbeit: Claude2026ah

```
279 def test_keine_ergebnisvariablen():
    ⋮
286     gefunden = [c for c in ERGEBNISVARIABLEN if c in klassifikation().columns]
287     assert not gefunden, f"Ergebnisvariable(n) im Datensatz: {gefunden}"
```

KURZ GESAGT **Der Test sucht die zehn gesperrten Felder in der fertigen Datei der Einsatzart.**

`ERGEBNISVARIABLEN` steht einmal in `config.py`, darunter Löschfahrzeuge, Alarmstufe und Ankunftszeit. Zeile 286 prüft die erzeugte Datei und nicht den Code, der sie schreibt. Gleich, wo ein Fehler läge, der Test schlägt an, sobald eines der Felder in der Datei steht.

CODE 3 · SEITE 31 DER ARBEIT

Zuordnung über Zensusgrenzen hinweg

prep/s1_daten.py · Z. 282–309, gekürzt · Quelle in der Arbeit: Claude2026ap

```
282 def tract_zu_stadtteil(geoids: pd.Series, crosswalk: pd.DataFrame) -> pd.Series:
303     direkt = dict(zip(crosswalk["geoid"], crosswalk["neighborhood"]))
304     basis = (crosswalk.assign(basis=crosswalk["geoid"].str[5:9])
305                       .groupby("basis")["neighborhood"]
306                       .agg(lambda s: s.iloc[0] if s.nunique() == 1 else None)
307                       .dropna().to_dict())
308     g = geoids.astype(str).str.zfill(11)
309     return g.map(direkt).fillna(g.str[5:9].map(basis))
```

KURZ GESAGT **Ein Tract findet seinen Stadtteil zuerst über die volle Kennung und nur wenn das scheitert über den vierstelligen Basiscode.**

Zeile 303 baut die direkte Zuordnung. Die Zeilen 304 bis 307 bilden je Basiscode einen Stadtteil, aber nur, wenn alle Nachfolger in denselben fallen (`nunique() == 1` in Zeile 306). Zeile 309 füllt mit `fillna` nur die Lücken der direkten Zuordnung und überschreibt nichts. Die Zuordnungsquote steigt damit auf 99,2 bis 99,5 % je Jahrgang.

CODE 4 · SEITE 32 DER ARBEIT

Zeitbewusste Zuordnung des ACS-Jahrgangs

prep/s1_daten.py · Z. 349–380, gekürzt · Quelle in der Arbeit: Claude2026q

```
349 def acs_snapshot(jahr: int, acs_years: list[int]) -> int:
364     return max([a for a in acs_years if a <= int(jahr) - ACS_PUBLIKATIONS_LAG],
365                default=min(acs_years))
380     sffd["acs_year"] = sffd["year"].apply(lambda y: acs_snapshot(y, jahrgaenge))
```

KURZ GESAGT **Ein Einsatz erhält den jüngsten ACS-Jahrgang, der zu seinem Zeitpunkt schon erschienen war.**

Zeile 364 lässt nur Jahrgänge bis Einsatzjahr minus `ACS_PUBLIKATIONS_LAG` zu und nimmt davon den größten. Ein Einsatz von 2020 erhält so den Jahrgang 2019, einer von 2023 den Jahrgang 2021. Zeile 380 wendet die Regel auf jede Einsatzzeile an. Der `default`-Zweig greift nur vor 2015 und kommt im Analysezeitraum nicht vor.

CODE 5 · SEITE 33 DER ARBEIT

Vollständiges Raster und Vorwärtsfüllen

prep/s2_datensaetze.py · Z. 222–234, gekürzt · Quelle in der Arbeit: Claude2026t

```
222 idx = pd.MultiIndex.from_product(
223     [agg["stadtteil"].unique(),
224      range(int(agg["jahr"].min()), int(agg["jahr"].max()) + 1),
225      range(1, 13)],
226     names=["stadtteil", "jahr", "monat"])
227 raster = agg.set_index(["stadtteil", "jahr", "monat"]).reindex(idx).reset_index()
228 raster[ZIELGROESSE] = raster[ZIELGROESSE].fillna(0).astype(int)
    ⋮
233 raster[_UEBERNOMMEN] = (raster.groupby("stadtteil")[_UEBERNOMMEN]
234                               .transform(lambda s: s.ffill()))
```

KURZ GESAGT **Jeder Stadtteil erhält alle 132 Monate, auch ohne Einsatz, und fehlende Merkmale werden nur nach vorn fortgeschrieben.**

Die Zeilen 222 bis 226 spannen das Raster aus Stadtteilen, Jahren und Monaten auf, `reindex` in Zeile 227 legt die Daten hinein. Eine leere Zielzelle heißt kein Einsatz und wird in Zeile 228 zur Null. Ein leeres Merkmal bleibt unbekannt, bis Zeile 233 den letzten bekannten Wert desselben Stadtteils übernimmt. Rückwärts zu füllen hieße, einen später erhobenen Wert einzuspielen.

CODE 6 · SEITE 34 DER ARBEIT

Anteilswerte aus Zählvariablen

prep/s1_daten.py · Z. 613–625, gekürzt · Quelle in der Arbeit: Claude2026r

```
613 def berechne_quoten(df: pd.DataFrame) -> pd.DataFrame:
    ⋮
622     for name, zaehler, nenner in QUOTEN:
623         z = pd.to_numeric(df[zaehler], errors="coerce").astype(float)
624         n = pd.to_numeric(df[nenner], errors="coerce").astype(float)
625         df[name] = (z / n.where(n > 0, np.nan)).round(4)
```

KURZ GESAGT **Alle Anteilsmerkmale entstehen in einer Schleife aus einer Liste von Zähler und Nenner.**

Die Liste `QUOTEN` legt je Anteil Zähler und Nenner fest, an einer einzigen Stelle. Zeile 625 lässt nur positive Nenner zu, ein Nenner von null ergibt einen fehlenden Wert statt null. `to_numeric` in den Zeilen 623 und 624 macht auch aus einem Platzhalter der Quelle einen fehlenden Wert.

CODE 7 · SEITE 35 DER ARBEIT

Relativer Kriminalitätsindex je Stadtteil und Monat

prep/s1_daten.py · Z. 508–530, gekürzt · Quelle in der Arbeit: Claude2026s

```
508 raster["delikte_fenster"] = (
509     raster.groupby("neighborhood")["delikte"]
510           .transform(lambda s: s.shift(1).rolling(CRIME_FENSTER_MONATE).sum()))
511 raster = raster.dropna(subset=["delikte_fenster"])
    ⋮
522 stadt = (raster.groupby(["jahr", "monat"])
523                .agg(stadt_delikte=("delikte_fenster", "sum"),
524                     stadt_bev=("total_population", "sum")).reset_index())
525 raster = raster.merge(stadt, on=["jahr", "monat"], how="left")
526 raster["crime_rate_raw"] = (raster["delikte_fenster"]
527                             / raster["total_population"] * 1000).round(3)
528 raster["crime_index"] = (raster["crime_rate_raw"]
529                         / (raster["stadt_delikte"] / raster["stadt_bev"] * 1000)
530                         ).round(4)
```

KURZ GESAGT **Der Index teilt die Deliktrate eines Stadtteils durch die der ganzen Stadt, beide aus den zwölf Monaten vor dem laufenden.**

Zeile 510 rückt die Reihe mit `shift(1)` um einen Monat zurück und summiert dann zwölf Monate. Der laufende Monat steckt also nie in seinem eigenen Merkmal. Die Zeilen 522 bis 524 bilden die Stadtsumme aus denselben Fenstern, die Zeilen 528 bis 530 teilen. Ein Wert von eins heißt Belastung wie im Stadtdurchschnitt, und ein Erfassungswechsel, der alle Stadtteile gleich trifft, kürzt sich heraus.

CODE 8 · SEITE 36 DER ARBEIT

Der Kalendermonat als Sinus und Kosinus

prep/s2_datensaetze.py · Z. 273–274 · Quelle in der Arbeit: Claude2026u

```
273 d["monat_sin"] = np.sin(2 * np.pi * d["monat"] / 12)
274 d["monat_cos"] = np.cos(2 * np.pi * d["monat"] / 12)
```

KURZ GESAGT **Der Monat geht als Punkt auf einem Kreis ein, damit Dezember und Januar Nachbarn sind.**

Als Zahl lägen Dezember und Januar elf auseinander. Der Sinus allein wäre nicht eindeutig, weil Januar und Mai denselben Wert erhielten, erst der Kosinus trennt sie. Die Zwölf im Nenner setzt die jährliche Periode.

CODE 9 · SEITE 39 DER ARBEIT

Prüfung der Stadtteiltrennung an der erzeugten Datei

`tests/test_aufbereitung.py` · Z. 136–158, gekürzt · Quelle in der Arbeit: Claude2026ah

```
136 def test_folds_ordnung_und_holdout():
    ⋮
144     d = regression()
145     holdout = set(d.loc[d["ist_holdout"] == 1, "stadtteil"])
146     gesehen = set()
147     for k in range(1, N_FOLDS + 1):
148         train, test = fold_masken(d, k)
149         st_train = set(d.loc[train, "stadtteil"])
150         st_test = set(d.loc[test, "stadtteil"])
151         assert st_train.isdisjoint(st_test), \
152             f"Fold {k}: Stadtteil in Training UND Test: {st_train & st_test}"
153         assert st_test.isdisjoint(holdout), f"Fold {k} greift ins Hold-out"
    ⋮
155         assert gesehen.isdisjoint(st_test), "Stadtteil in zwei Folds im Test"
156         gesehen |= st_test
157     assert gesehen | holdout == set(d["stadtteil"]), \
158         "Nicht jeder Stadtteil ist genau einmal Testfall oder im Hold-out"
```

KURZ GESAGT **Der Test baut die fünf Folds aus der Datei nach und prüft, dass kein Stadtteil zugleich Trainings- und Testfall ist.**

Zeile 151 verlangt getrennte Stadtteile in Training und Test, Zeile 153 verbietet dem Testfold jeden Hold-out-Stadtteil und Zeile 155 einen Stadtteil in zwei Testfolds. Zeile 157 verlangt Vollständigkeit: Jeder Stadtteil ist genau einmal Testfall oder im Hold-out. Die Masken kommen aus `fold_masken()`, derselben Funktion, die auch die Modelle benutzen.

CODE 10 · SEITE 40 DER ARBEIT

Fold-Zuteilung mit doppelter Stratifizierung

`prep/s2_datensaetze.py` · Z. 62–94, gekürzt · Quelle in der Arbeit: Claude2026v

```
62 def ergaenze_aufteilung(daten: pd.DataFrame, versatz: int = 0,
63                         selten: pd.Series | None = None) -> pd.DataFrame:
    ⋮
81     bev = daten.groupby("stadtteil")[EXPOSURE_ROH].mean()
    ⋮
84     ordnung = (pd.DataFrame({"selten": selten.reindex(bev.index).fillna(0),
85                              "bev": bev})
86                  .sort_values(["selten", "bev"], ascending=False).index)
    ⋮
89     gruppe = {st: (i + versatz) % (N_FOLDS + 1) for i, st in enumerate(ordnung)}
    ⋮
91     d = daten.copy()
92     d["fold"] = d["stadtteil"].map(gruppe).astype(int)
93     d["ist_holdout"] = (d["fold"] == 0).astype(int)
94     return d
```

KURZ GESAGT **Die Stadtteile werden nach Brandmonaten und Größe sortiert und reihum auf sechs Gruppen verteilt. Die erste Gruppe ist das Hold-out.**

Die Zeilen 84 bis 86 sortieren ohne Zufall, erst nach der Zahl brand-dominierter Monate, dann nach Wohnbevölkerung. Zeile 89 verteilt reihum, Zeile 93 macht Gruppe 0 zum Hold-out. Weil Folds und Hold-out aus derselben Zuteilung kommen, können sie nicht auseinanderlaufen. Der brandreichste Stadtteil steht dabei an erster Stelle und fällt so ins Hold-out.

CODE 11 · SEITE 42 DER ARBEIT

Mittelwert und Streuung der Gütemaße über Folds und Wiederholungen

modelle/m02_menge.py · Z. 437–454, gekürzt · Quelle in der Arbeit: Claude2026ae

```
437 def aggregiere(folds: pd.DataFrame) -> pd.DataFrame:
    ⋮
448     schluessel = ["zielgroesse", "verfahren"]
449     g = folds.groupby(schluessel, sort=False)
450     z = g[MASSE].mean().add_suffix("_mean")
451     z = z.join(g[MASSE].std().add_suffix("_std_folds"))
452     je_wdh = folds.groupby(schluessel + ["wiederholung"], sort=False)[MASSE].mean()
453     z = z.join(je_wdh.groupby(schluessel, sort=False).std()
454                       .add_suffix("_std_wiederholungen"))
```

KURZ GESAGT **Der Mittelwert kommt aus allen 50 Läufen, die maßgebliche Streuung aus den zehn Wiederholungsmitteln.**

Zeile 450 mittelt direkt, bei fünf Folds je Wiederholung ist das dasselbe wie zweistufig. Zeile 451 bildet die Streuung über alle 50 Läufe, die Zeilen 452 bis 454 erst das Mittel je Wiederholung und dann die Streuung dieser zehn Werte. Beide Spalten stehen in der Ergebnisdatei. Die 50 Läufe sind dieselben 30 Stadtteile in zehn Gruppierungen und damit nicht unabhängig.

CODE 12 · SEITE 43 DER ARBEIT

Der Offset im Poisson-Modell der Stufe 2

vorpruefung/v1_baselines.py · Z. 74–108, gekürzt · Quelle in der Arbeit: Claude2026x

```
74 def poisson_glm(train: pd.DataFrame, test: pd.DataFrame,
75                 merkmale: list[str] | None = None) -> np.ndarray:
    ⋮
95     spalten = MERKMALE if merkmale is None else list(merkmale)
96     X_tr = sm.add_constant(train[spalten].astype(float), has_constant="add")
97     X_te = sm.add_constant(test[spalten].astype(float),  has_constant="add")
98     y_tr = train[ZIELGROESSE].astype(float)
    ⋮
102    off_tr = np.log(train[EXPOSURE_ROH].astype(float))
103    off_te = np.log(test[EXPOSURE_ROH].astype(float))
    ⋮
106    modell = sm.GLM(y_tr, X_tr, family=sm.families.Poisson(),
107                    offset=off_tr).fit()
108    return np.asarray(modell.predict(X_te, offset=off_te))
```

KURZ GESAGT **Das Poisson-Modell schätzt eine Rate, weil die logarithmierte Wohnbevölkerung als Offset mit festem Koeffizienten eins eingeht.**

Die Zeilen 102 und 103 bilden den Offset getrennt für Trainings- und Teststadtteile. Zeile 107 gibt ihn der Anpassung mit, Zeile 108 der Vorhersage. Ohne den zweiten Aufruf käme eine Rate ohne Bezug zur Einwohnerzahl des Teststadtteils heraus. Zeile 106 trägt weder Strafterm noch Suche, das Modell hat keinen Hyperparameter.

CODE 13 · SEITE 43 DER ARBEIT

Beide Baseline-Stufen des Mengenstrangs

vorpruefung/v1_baselines.py · Z. 141–171, gekürzt · Quelle in der Arbeit: Claude2026x

```
141 def regression(panel: pd.DataFrame, selten: pd.Series) -> pd.DataFrame:
154     for w in range(WIEDERHOLUNGEN):
155         d = wiederholte_aufteilung(panel, wiederholung=w, selten=selten)
156         for k in range(1, N_FOLDS + 1):
158             train, test = d[tr], d[te]
159             bev = test[EXPOSURE_ROH].to_numpy()
161             anzahl = poisson_glm(train, test)
163             for ziel, referenz in ((ZIELGROESSE, anzahl),
164                                    (RATE, anzahl / bev * 1000)):
166                 for stufe, name, y_hat in (
167                         (1, NULLMARKE, np.full(len(test), train[ziel].mean())),
168                         (2, POISSON, referenz)):
169                     zeilen.append({"wiederholung": w, "fold": k, "stufe": stufe,
```

KURZ GESAGT **Beide Baseline-Stufen und beide Mengenzielgrößen entstehen in einem Lauf über dieselben 50 Aufteilungen wie die Verfahren.**

Zeile 154 zählt die zehn Wiederholungen, Zeile 156 die fünf Folds. Zeile 161 passt das Poisson-Modell einmal je Fold an, Zeile 164 leitet die Rate aus derselben Vorhersage ab. Die Zeilen 166 bis 169 schreiben Stufe 1, den Mittelwert der Trainingsstadtteile, und Stufe 2 nebeneinander in dieselbe Datei.

CODE 14 · SEITE 48 DER ARBEIT

Überdispersionstest als Hilfsregression

vorpruefung/v2_eignung.py · Z. 403–415, gekürzt · Quelle in der Arbeit: Claude2026y

```
403 X = sm.add_constant(train[MERKMALE].astype(float), has_constant="add")
404 y = train[ZIELGROESSE].astype(float).to_numpy()
410 poisson = sm.GLM(y, X, family=sm.families.Poisson(),
411                  offset=np.log(train[EXPOSURE_ROH].astype(float))).fit()
412 mu = np.asarray(poisson.fittedvalues, float)
413 z = ((y - mu) ** 2 - y) / mu
414 hilfs = sm.OLS(z, mu).fit()
415 ct_t, ct_p = float(hilfs.tvalues[0]), float(hilfs.pvalues[0]) / 2
```

KURZ GESAGT **Der Test fragt, ob die Varianz der Einsatzzahlen über ihrem Erwartungswert liegt, mit einer Hilfsregression nach Cameron und Trivedi.**

Zeile 410 passt dasselbe Poisson-Modell mit Offset an wie die Referenz. Zeile 413 bildet eine Hilfsgröße, die im Mittel null ist, solange Varianz und Erwartungswert übereinstimmen. Zeile 414 regressiert sie ohne Konstante auf die Vorhersage, Zeile 415 halbiert den p-Wert für den einseitigen Test. Mit t = 18,0 ist die Annahme verletzt. Das betrifft die Standardfehler, nicht die Punktvorhersage.

CODE 15 · SEITE 51 DER ARBEIT

Ein Lauf: Anpassung auf der Rate, Rückrechnung und Zeitmessung

modelle/m02_menge.py · Z. 219–232, gekürzt · Quelle in der Arbeit: Claude2026aa

```
219 auf_rate = ziel == ZIELGROESSE
220 y_tr = train[RATE if auf_rate else ziel].astype(float)
221 zurueck = (test[EXPOSURE_ROH].astype(float).to_numpy() / 1000.0
222            if auf_rate else 1.0)
224 modell = verfahren(name).set_params(**parameter)
226 t = time.perf_counter()
227 modell.fit(X_tr, y_tr)
228 train_sek = time.perf_counter() - t
230 t = time.perf_counter()
231 y_hat = modell.predict(X_te) * zurueck
232 inferenz_sek = time.perf_counter() - t
```

KURZ GESAGT **Alle Verfahren lernen die Rate je 1.000 Einwohner und rechnen erst die fertige Vorhersage auf die Einsatzzahl zurück.**

Zeile 219 setzt `auf_rate` bei der Anzahl auf wahr, Zeile 220 nimmt in beiden Fällen die Rate als Trainingsziel. Die Zeilen 221 und 222 wählen den Faktor für die Rückrechnung, Zeile 231 wendet ihn nach `predict` an. Die Stoppuhr umschließt in den Zeilen 226 bis 228 nur die Anpassung und in 230 bis 232 nur die Vorhersage.

CODE 16 · SEITE 53 DER ARBEIT

Verlustfunktion und Zieltransformation je Verfahren

modelle/m02_menge.py · Z. 81–115, gekürzt · Quelle in der Arbeit: Claude2026aa

```
 81 def verfahren(name: str, n_jobs: int = N_JOBS_MODELL):
110     if name == "ridge": return
    make_pipeline(StandardScaler(),TransformedTargetRegressor(regressor=Ridge(),func=np.log1p,
    inverse_func=np.expm1))
111     if name == "random_forest":
112         return RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=n_jobs,
    criterion="poisson")
113     if name == "xgboost":
114         from xgboost import XGBRegressor
115         return XGBRegressor(random_state=RANDOM_STATE, n_jobs=n_jobs,objective="reg:tweedie")
```

KURZ GESAGT **Alle drei Verfahren entstehen in einer Funktion, jedes mit einer Verlustfunktion, die zu Zähldaten passt.**

Ridge rechnet in Zeile 110 auf log(1+y) und transformiert mit `expm1` zurück. Standardisierung und Transformation sitzen in der Pipeline und sehen je Fold nur die Trainingszeilen. Der Random Forest minimiert in Zeile 112 die Poisson-Devianz, XGBoost in Zeile 115 die Tweedie-Devianz mit gesuchtem Exponenten. Beide Bäume holen ihren Startwert aus `RANDOM_STATE`.

CODE 17 · SEITE 54 DER ARBEIT

Globale Klassenkodierung und ausgeglichene Gewichte

modelle/m03_struktur.py · Z. 134–158, gekürzt · Quelle in der Arbeit: Claude2026ab

```
134 def kodiere(y: pd.Series) -> np.ndarray:
    ⋮
145     index = {k: i for i, k in enumerate(KLASSEN)}
146     unbekannt = set(y.unique()) - set(index)
147     assert not unbekannt, f"Unbekannte Klassen im Datensatz: {unbekannt}"
148     return y.map(index).to_numpy()
    ⋮
151 def _gewichte(y_int: np.ndarray) -> np.ndarray:
    ⋮
158     return compute_sample_weight("balanced", y_int)
```

KURZ GESAGT **Die vier Klassen erhalten einmal für den ganzen Datensatz feste Nummern, die Gewichte entstehen je Fold.**

Zeile 145 baut die Nummern aus der Konstante `KLASSEN` und nicht aus den Daten eines Folds. Fehlt in einem Fold die Brandklasse, verschieben sich die Nummern also nicht. Die Zeilen 146 und 147 brechen bei einer unbekannten Klasse ab. Zeile 158 gewichtet umgekehrt zur Häufigkeit im Trainingsteil, keine Zeile wird vervielfacht oder gelöscht.

CODE 18 · SEITE 54 DER ARBEIT

Innere Suche, nach Stadtteilen gruppiert

modelle/m02_menge.py · Z. 155–184, gekürzt · Quelle in der Arbeit: Claude2026ad

```
155 def tune(name: str, train: pd.DataFrame, ziel: str) -> dict:
    ⋮
173     suche = RandomizedSearchCV(
174         estimator=verfahren(name, n_jobs=N_JOBS_MODELL),
175         param_distributions=suchraum(name),
176         n_iter=TUNING_BUDGET,
177         cv=GroupKFold(n_splits=4),
178         scoring="neg_root_mean_squared_error",
179         random_state=RANDOM_STATE,
180         n_jobs=N_JOBS_SUCHE,
181     )
182     suche.fit(train[MERKMALE].astype(float), train[ziel].astype(float),
183               groups=train["stadtteil"])
184     return suche.best_params_
```

KURZ GESAGT **Die Suche zieht 100 Kombinationen und prüft sie in einer inneren Kreuzvalidierung, die ebenfalls nach Stadtteilen trennt.**

Zeile 177 nimmt `GroupKFold` mit vier Folds, Zeile 183 übergibt die Stadtteile als Gruppen. Ohne beides läge derselbe Stadtteil in innerem Training und innerer Validierung. Zeile 180 verteilt die Ziehungen auf alle Kerne, das Modell selbst rechnet einkernig (Zeile 174). Zeile 184 gibt nur die Parameter zurück und nicht das Modell.

CODE 19 · SEITE 55 DER ARBEIT

Der Bewertungslauf über alle Wiederholungen und Folds

modelle/m02_menge.py · Z. 388–415, gekürzt · Quelle in der Arbeit: Claude2026ae

```
388 def phase_bewertung(panel: pd.DataFrame, parameter: pd.DataFrame,
389                     selten: pd.Series) -> pd.DataFrame:
    ⋮
400     param = _parameter_je_fold(parameter)
401     zeilen, vorhersagen = [], []
402     for w in range(WIEDERHOLUNGEN):
403         d = wiederholte_aufteilung(panel, wiederholung=w, selten=selten)
404         for k in range(1, N_FOLDS + 1):
405             tr, te = fold_masken(d, k)
406             train, test = d[tr], d[te]
407             for ziel in ZIELE:
408                 for name in VERFAHREN:
    ⋮
410                     z = ein_lauf(name, param[(ziel, name, k)], train, test,
411                                  ziel, auch_parallel=True,
412                                  mit_vorhersagen=True)
413                     vorhersagen.append(z.pop("_vorhersagen")
414                                        .assign(wiederholung=w, fold=k))
415                     zeilen.append({"wiederholung": w, "fold": k, **z})
```

KURZ GESAGT **Vier verschachtelte Schleifen ergeben die 300 Läufe des Mengenstrangs.**

Zehn Wiederholungen (Zeile 402), fünf Folds (Zeile 404), zwei Zielgrößen (Zeile 407) und drei Verfahren (Zeile 408). Zeile 410 baut mit den gesuchten Parametern ein frisches Modell auf allen Trainingsstadtteilen des Folds. `auch_parallel=True` in Zeile 411 misst jeden Lauf zusätzlich über alle Kerne. Berichtet wird der einkernige Lauf, weil XGBoost mehrkernig um bis zu 50,88 Einsätze abweicht.

WAS ALLE STELLEN VERBINDET

Leckage wird an der Quelle verhindert. Merkmale schauen nur zurück, Folds und Hold-out stehen als Spalten in der Datei, und jede Suche trennt nach Stadtteilen.

Fairness ist eine Eigenschaft des Datensatzes. Alle Verfahren lesen dieselben zwölf Merkmale, dieselben Zeilen und dieselben Folds, und die Referenz läuft unter demselben Protokoll mit.

Die Statistik rechnet mit der Information, die wirklich da ist. Getestet wird auf zehn Wiederholungsmitteln statt auf 50 Läufen, und ein nicht signifikanter Abstand gilt als nicht trennbar und nicht als Gleichheit.